

Large Language Model Lineage Analysis: How far are we?

Anonymous Author(s)

Abstract

CCS Concepts

• **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

Anonymous Author(s). 2026. Large Language Model Lineage Analysis: How far are we?. In . ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

2 Motivation

The proliferation of Large Language Models (LLMs) has been dramatically accelerated by collaborative platforms like the [Hugging Face Hub](#), which serves as a central repository for model sharing. A cornerstone of this ecosystem is the iterative development of new models by building upon existing ones through techniques such as fine-tuning, merging, and quantization. This process creates a complex web of inter-model dependencies, which we define as **model genealogy** or **lineage**. A clear understanding of a model's genealogy is paramount for ensuring traceability, reproducibility, and compliance, as derivative models often inherit biases, vulnerabilities, or licensing constraints from their predecessors.

However, the mechanisms currently available on the Hugging Face Hub for documenting this critical lineage information are frequently inconsistent, incomplete, or entirely absent. This lack of a systematic and enforceable standard for provenance documentation poses a significant barrier to researchers and developers. The most visible symptom of this issue is the difficulty, or outright impossibility, of programmatically constructing a model's lineage—a structure ideally represented as a “Model Tree”. This is not a superficial interface problem but a direct consequence of fundamental flaws in how provenance data is recorded at its source.

To illustrate the severity of this challenge, we will dissect two distinct modes of documentation failure, each representing a progressively worsening degree of information obfuscation.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

2.1 Problem 1: Lineage Concealed in Unstructured Text

In the most common scenario, lineage information exists but is locked within free-form text, rendering it inaccessible to automated tools. This forces researchers into a manual, error-prone process of discovery.

Case Study 1: DavidAU/NSFW_DPO_Noromaid-7b-GGUF

The Hugging Face repository of this model does not include a model tree; however, its model card explicitly states:

NSFW_DPO_Noromaid-7b-Mistral-7B-Instruct-v0.1 is a merge of the following models: mistralai/Mistral-7B-Instruct-v0.1 and athirdpath/NSFW_DPO_Noromaid-7b.

Notably, the Hugging Face pages of both referenced models in the model card contain their respective model trees. This reliance on unstructured documentation makes the automated construction of a lineage graph infeasible. It imposes a significant manual burden for identifying even a direct parent and completely obstructs any large-scale, systematic analysis of the model ecosystem.

Case Study 2: timdettmers/guanaco-13b

Another example is the timdettmers/guanaco-13b model. While this model is a fine-tuned version of LLaMA, this pivotal relationship is not captured in any structured, machine-readable metadata. To discover this link, a user must manually navigate to the model's repository, open the “Model Card” (a Markdown file), and parse through natural language prose to find the statement:

Guanaco models are open-source finetuned chatbots obtained through 4-bit QLoRA tuning of LLaMA base models...

2.2 Problem 2: Lineage Outsourced to Fragile External Links

A more precarious situation arises when provenance information is not even located within the Hugging Face platform, but is instead hyperlinked to an external, third-party source.

Case Study 3: pfnnet/plamo-2.1-2b-cpt

The model’s homepage lacks a model tree, and its model card merely states that the model was derived by reducing the parameter size from an 8B-parameter model, without explicitly specifying the base model used for this operation. However, the model card provides links to three technical blogs for reference. By following these external links, we can infer that the mentioned 8B-parameter model is plano-2-8b.

Case Study 4: Remilistrasza/Mysterious_SDXL_Lah

Consider the Remilistrasza/Mysterious_SDXL_Lah model. Its Model Card is starkly empty of descriptive content, offering only a single hyperlink to a page on Civitai, another model-sharing platform. To identify the base model, a user is forced to leave the Hugging Face ecosystem. Only upon navigating to this external page does one discover that the base model is “SDXL 1.0”.

This practice introduces two severe risks: **information fragmentation** and **link rot**. The integrity and availability of the lineage information become wholly dependent on the uptime and archival policies of an external website. Should the link become invalid (a 404 error), or the content on the remote page be altered or deleted, the model’s traceable lineage is permanently severed. This practice renders the model’s provenance fragile and unverifiable from within the Hugging Face ecosystem.

These documented failures, which range from information concealed in prose to its complete omission, demonstrate a clear and pressing need for an automated and reliable method to construct model genealogies. The current reliance on inconsistent and manual documentation is an unsustainable bottleneck in an ecosystem that is scaling rapidly. To bridge this critical gap, we propose an approach to automatically identify and verify the derivative relationships between models, thereby enabling robust and scalable lineage tracing.

3 Benchmark

3.1 Construction

The construction of our benchmark involves a systematic pipeline that traces model derivations from base models to their descendants on Hugging Face. This process is divided into five key steps:

Step 1: Obtaining Base Model List. We begin by collecting a list of popular base models for the text-generation task from Hugging Face. Utilizing the HfApi provided by huggingface_hub, we retrieve the top 50 non-gated models sorted by the number of likes. The model identifiers are stored in a file named base_models.txt, where each line corresponds to one base model ID. This step is implemented via **Script 1**.

Step 2: Building the Model Derivation Tree. Based on base_models.txt, we recursively crawl derivative models to construct a “model forest”. For each base model, we explore its derivatives up to four

levels deep, covering four derivation types: finetune, adapter, quantized, and merge. For each derivation type, a maximum of six sibling nodes are considered. Every node in the tree records metadata such as model name, crawl timestamp, depth level, and its child nodes. The resulting forest structure is serialized into model_forest.json. This step is implemented via **Script 2**.

Step 3: Constructing Model Pairs. From the model forest, we extract two categories of model pairs: adjacent parent-child pairs and non-adjacent ancestor-descendant pairs. By performing a Breadth-First Search (BFS) traversal on each tree, we generate:

- **Adjacent Pairs:** Direct parent-child model pairs.
- **Non-Adjacent Pairs:** All ancestor-descendant pairs where the ancestor is not the immediate parent.

These pairs are recorded in adjacent_pairs.jsonl and non_adjacent_pairs.jsonl respectively. This step is implemented via **Script 3**.

Step 4: Single-Threaded Model Validation. To ensure data quality, we validate the structural correctness of all adjacent model pairs. For each model, we inspect its file tree on Hugging Face, checking for the presence of critical files such as config.json, adapter_config.json, and various parameter files (e.g., .bin, .pt, .safetensors, etc.). We verify whether the claimed derivation type aligns with the actual file structure and whether the parent model is valid. The results are compiled into a comprehensive validation report named model_validation_report.json. This step is implemented via **Script 4**.

Step 5: Filtering Erroneous Model Pairs. Based on the validation report, we remove all erroneous model pairs from the dataset. Models flagged with any inconsistencies or missing critical files are excluded. The cleaned set of adjacent model pairs is saved in adjacent_pairs_without_error.json. This step is implemented via **Script 5**.

3.2 Characteristics

Our benchmark exhibits several notable characteristics that reflect its diversity and representativeness of the Hugging Face model ecosystem:

- **Variety of Derivation Types:** The dataset covers four major derivation types—finetune, adapter, quantized, and merge—capturing a wide range of model transformation patterns.
- **Hierarchical Depth:** With a recursive crawling depth of up to four levels, the benchmark contains both shallow and deep derivation chains, enabling analysis of both immediate and long-range model inheritance.
- **Scale and Coverage:** Starting from 50 popular base models, the recursive traversal results in a substantial model forest, ensuring coverage of highly influential models as well as their derivative variants.
- **Data Cleanliness:** Through rigorous validation and error filtering, the dataset ensures high structural integrity, making it suitable for tasks that require reliable model lineage information.

Table 1: Statistics of the GRANDMoTHER benchmark.

Statistic	Value
#Graphs	31
#Model pairs (edges)	248
Min / Median / Max pairs per graph	1 / 6 / 22
Avg. pairs per graph	8.0
#Models (nodes)	279
#Root models	31
#Intermediate models	44
#Leaf models	204
Share of leaf models	73.1%
Min / Avg / Max models per graph	2 / 9.0 / 23
Avg. out-degree of non-leaf models	3.31
Max fine-tuning depth	2

Table 2: Lineage Detection Accuracy (%) w.r.t. Different Measurement Metrics, Direction Methods, and Graph Algorithms (RQ1)

Graph Algorithm	Dir.	Measurement Metric					
		L2	Cos.	AM	AMx	SA	SR
+ MoTHER	K	...	...	...	...	...	...
	T	...	...	...	...	...	...
+ Hard-Constraint	K	...	...	...	...	...	...
	T	...	...	...	...	...	...
+ Undirected-Mst	K	...	...	...	...	...	...
	T	...	...	...	...	...	...
+ atlas_snake_fan	K	...	...	...	...	...	...
	T	...	...	...	...	...	...

- **Real-World Relevance:** The benchmark reflects actual user-driven derivation patterns on Hugging Face, providing insights into community practices such as model fine-tuning trends, adapter usage, and quantization adoption.

4 Empirical Setup

We design experiments to explore the following research questions.

- **Measurement Evaluation (RQ1):** How do different measurement metrics affect the effectiveness of LLM lineage detection?
- **Layers in Transformer Block Evaluation (RQ2):** How do different model layers influence the effectiveness of LLM lineage detection?
- **Failure Analysis (RQ3):** Why do existing methods fail to detect lineage?
- **Integration (RQ4):** using the above findings, the best performance can achieve? (80%)

Measurement Metrics. XXX

Accuracy.

Layers. XXX

Graph Algorithms. XX

5 RQ1: Measurement Evaluation

Summary: L2+timestamp+xxgraph algorithm performs best.

6 RQ2: Layers in Transformer Block Evaluation

Summary: using QKV layer performs best. K layer contributes most.

Summary:

7 RQ3: Failure Analysis

Multi-hop Analysis (RQ3): XX

8 Related Work

8.1 Model Signature and Watermark

The commercial value and immense training costs of deep learning models have established them as critical intellectual property (IP), prompting extensive research into model watermarking and provenance tracking. Early defense mechanisms primarily targeted discriminative neural networks to thwart model extraction and surrogate attacks [18, 110, 121]. These methods established ownership by verifying embedded external features or backdoor triggers in a black-box setting [48, 49]. More proactive defenses, such as Bayesian active watermarking, have since been developed to actively inject traceability during user queries, mitigating the risk of unauthorized model cloning across various paradigms, including federated learning [7, 87, 99].

As the AI landscape shifted towards generative architectures, adversaries began exploiting unauthorized fine-tuning and task adaptation to bypass traditional IP protections. Consequently, visual and multi-modal watermarking evolved to embed robust, structural fingerprints that survive downstream model modifications. To comprehensively protect these diverse generative assets, researchers have embedded invisible, verifiable signatures across a wide spectrum of generation processes, encompassing latent and pixel-space diffusion models [15, 24, 31, 52, 85, 88, 100], as well as emerging audio and video synthesis frameworks [30, 38, 53, 123]. Moving beyond mere generation tracking to combat model adaptation, techniques like SleeperMark [86] explicitly disentangle watermark information from semantic concepts in text-to-image diffusion models, ensuring the model retains its embedded fingerprint even after substantial downstream fine-tuning. Similar structural resilience has been integrated into diverse architectures, spanning GANs [14, 50], 3D scene representations like NeRFs and 3D Gaussian Splatting [10, 37, 56], and pivotal tuning processes for pre-trained image generators [55]. However, the robustness of these continuous-domain watermarks is continually challenged by universal forgery and erasure attacks [42, 60].

In the domain of Large Language Models (LLMs), watermarking initially emerged as a mechanism to detect machine-generated text via vocabulary partitioning, typically by promoting a randomized “green list” of tokens during decoding [43]. Subsequent research heavily optimized these foundational methods to enhance text quality, information capacity, and statistical robustness [8, 16, 17, 28, 35, 39, 54, 67, 70, 81, 90, 103], alongside advanced algorithms for segmenting and detecting partial watermarks [46, 116]. Concurrently, adversaries have developed sophisticated evasion techniques, including color-aware substitution, paraphrasing, and cross-lingual attacks [25, 41, 84, 89]. However, as the primary threat landscape evolved from content misuse to intellectual property

Ablation Study: Impact of Q/K/V/O/MLP on Accuracy

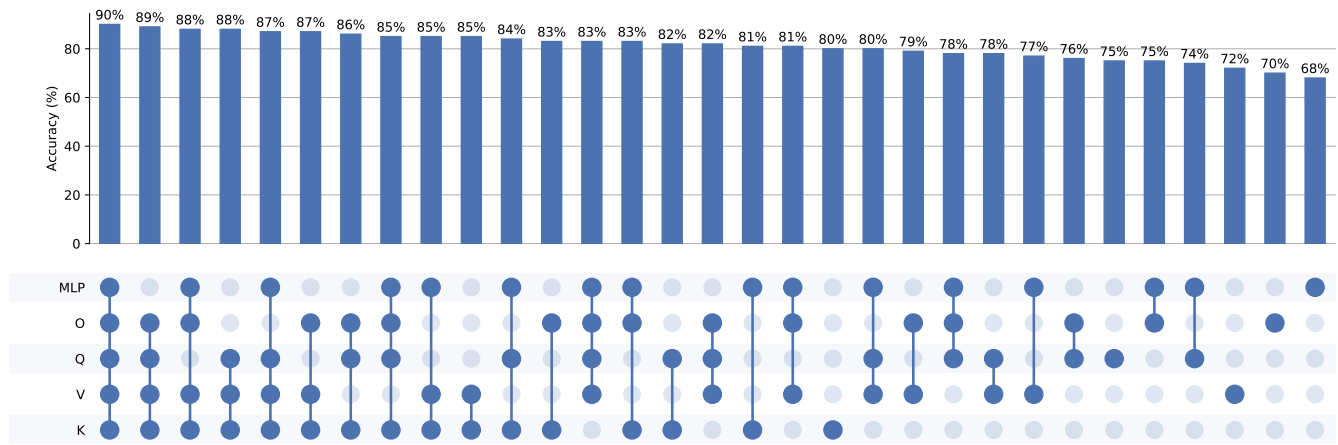


Figure 1: Accuracy of lineage detection with different combinations of transformer block components (RQ2).

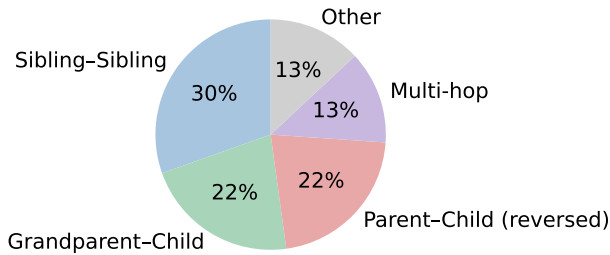
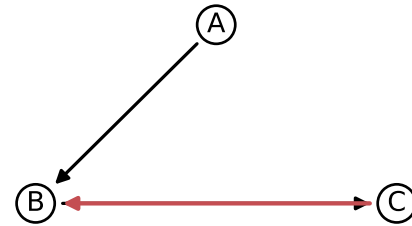


Figure 2: Proportion of different types of erroneous edges in the constructed lineage graph (RQ3).

theft via knowledge distillation, researchers pivoted to adapt these text-tracking techniques for proprietary model protection. Building upon the statistical manipulation of decoding probabilities, specialized black-box frameworks such as PLMmark [45] and GINSEW [117] were developed to detect unauthorized student models that inherit the behavioral fingerprints of the protected teacher model.

Beyond simple binary detection, securing model provenance requires addressing critical cryptographic vulnerabilities such as key leakage and malicious framing. To this end, researchers have introduced unforgeable, publicly verifiable watermarking schemes (UPV)



Red arrow: erroneous sibling edge (C → B)

Figure 3: Case study illustrating erroneous edges in the constructed lineage graph (RQ3).

that utilize dual neural networks to decouple the generation and detection keys [51]. Furthermore, defending against spoofing attacks—where adversaries forge harmful content to misattribute blame to the model owner—has driven the development of bi-level signature schemes like Bileve [119]. Advancing from model-level to user-level provenance, recent frameworks have also enabled multi-user watermarking, allowing developers to trace unauthorized model extraction or data leakage back to individual colluding users even under adaptive prompting [11]. Collectively, these advancements provide a robust mechanism to trace model lineage and detect unauthorized distillation across the AI supply chain [63, 64, 72]. To achieve holistic ecosystem protection, tracking mechanisms are also being aggressively extended to safeguard peripheral, high-value assets such as training data, source code, and prompts [77, 96, 101].

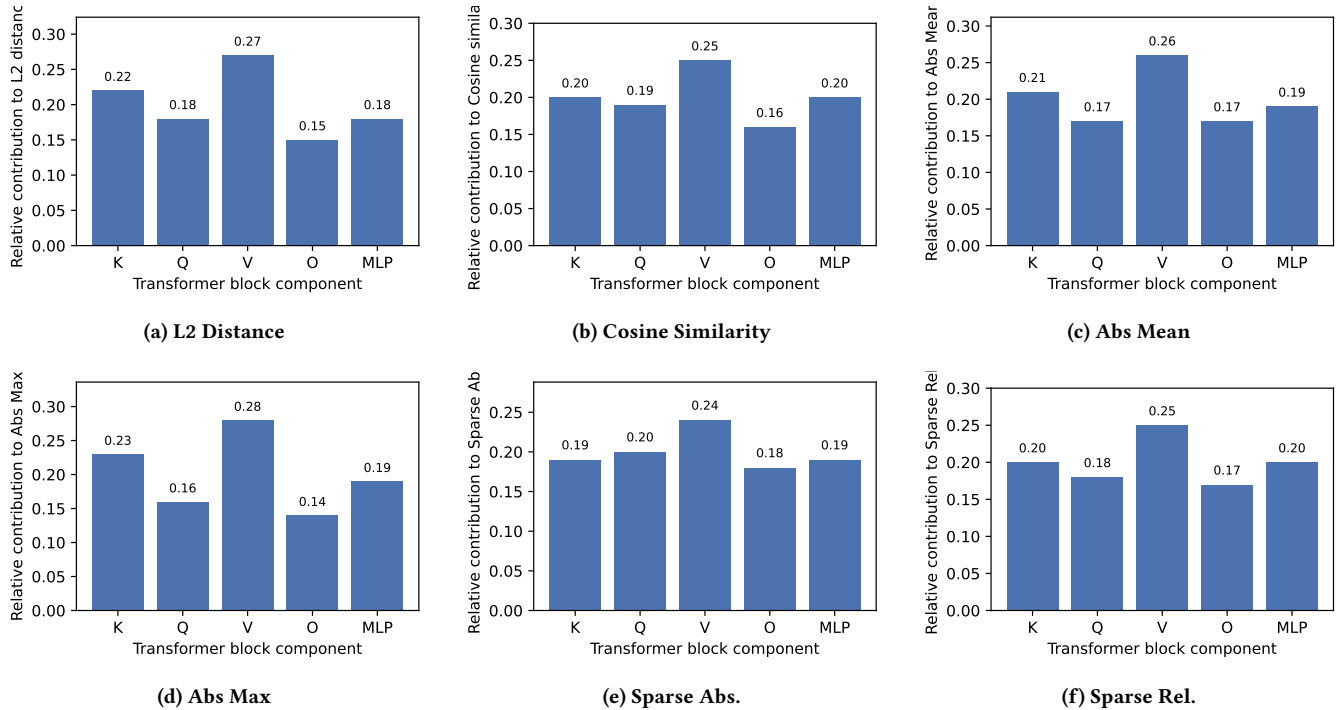


Figure 4: Contribution of different transformer block components (K/Q/V/O/MLP) to various distance metrics (RQ2).

These dynamic arms races are comprehensively evaluated in recent benchmarks and surveys [71, 79, 115, 118].

8.2 Model Fingerprinting and Similarity

Beyond active watermarking, model fingerprinting seeks to identify the unique, inherent characteristics of models to track their provenance and verify ownership. These techniques span a spectrum from black-box behavioral profiling to white-box static analysis. In black-box scenarios, fingerprinting relies on analyzing a model’s dynamic inference behavior in response to carefully crafted inputs. For discriminative models, this involves analyzing output correlation statistics on specific samples or learning the subspace of universal adversarial perturbations [22, 66, 98]. As Large Language Models (LLMs) became prevalent, these dynamic probing techniques evolved to handle complex deployments. Researchers developed methods to actively query integrated applications to identify the underlying LLM engine [65], or constructed specific prompts which often use adversarial suffixes or dual-layered stylistic and semantic triggers to reliably elicit unique behavioral signatures even after the model has undergone fine-tuning or merging [23, 94, 95]. Furthermore, analyzing the output distributions and log-likelihood vectors across standardized task suites allows for the construction of functional model DNA or phylogenetic maps, enabling the clustering and lineage tracking of hundreds of black-box LLMs [62, 91, 102].

Transitioning to white-box settings, researchers exploit access to internal model states to derive highly robust fingerprints without relying solely on black-box output distributions. Initial white-box approaches evaluated the similarity of internal representations, gradient tensors, and activation differences to establish familial links [92, 93, 111]. Moving towards completely static offline analysis,

recent advancements delve entirely into the parameter space, extracting intrinsic signatures directly from the model weights. These purely static analyses include leveraging the statistical distributions of attention parameters [104], identifying directionally invariant terms within the weights to generate verifiable ownership proofs [109], and utilizing singular value decomposition to extract invariant spectral signatures from attention matrices for lineage tracking [82]. Furthermore, rigorous statistical methods evaluating weight matrix pairs have been introduced to specifically detect unauthorized weight copying and plagiarism [112]. Building upon these offline parameter measurements, unsupervised algorithms have also been proposed to reconstruct precise evolutionary model trees, a process achieved by formulating lineage recovery as a directed minimum spanning tree problem based on the L_2 distances between model weights [27].

8.3 Layer-wise Dynamics in Fine-tuning

Understanding the impact of fine-tuning on pre-trained language models requires examining both macroscopic representation spaces and microscopic parametric mechanisms. At the macroscopic level, fine-tuning adjusts layer-wise hidden states to efficiently extract task-specific features without catastrophically altering the foundational semantic capabilities of early layers [80]. This adaptation alters the global embedding geometry primarily by expanding inter-class margins while preserving the original spatial topology of the pre-trained data [120]. Microscopically, these representation shifts are governed by specific structural modules, particularly feed-forward networks (FFNs) acting as hierarchical key-value memories [21]. The adaptation process within these layers is highly localized;

modifying distinct FFN sub-layers yields divergent knowledge updating dynamics [68], and specific facts are often sparsely encoded in distinct “knowledge neurons” that allow for targeted interventions [12]. Ultimately, the layer-wise adaptation during fine-tuning exhibits an intrinsic rank-deficiency, demonstrating that these complex weight updates can be robustly approximated by injecting trainable low-rank matrices into frozen Transformer layers [29]. These layer-wise insights provide the theoretical foundation for tracing model lineage during downstream adaptation.

8.4 Model Lineage Analysis

Building upon model fingerprinting and pairwise similarity techniques, recent research has advanced towards systematically reconstructing the evolutionary lineage of neural networks. While earlier methods primarily focused on calculating undirected distances or pair-wise similarities—such as using centered kernel alignment [111], gradient-based fingerprints [92], matrix-level homology [112], or log-likelihood vectors [62]—to cluster models or detect unauthorized derivations, the current frontier emphasizes the construction of directed lineage trees and graphs. For instance, to map the opaque landscape of fine-tuned and distilled models, researchers have adapted phylogenetic algorithms and defined low-dimensional functional “LLM DNA” based on model outputs to build evolutionary trees and dendrograms [91, 102]. In the white-box domain, neural lineage detection has been formalized to explicitly predict parent-child tuning relationships [105], and unsupervised algorithms have been designed to decode directed minimal spanning trees from model weights by leveraging weight kurtosis as a proxy for evolutionary direction [27]. Ultimately, these methodologies converge toward the broader vision of a “Model Atlas”, a unified graph charting the developmental trajectories and weight transformations of the global model population, often utilizing release metadata and timestamps to anchor temporal lineage [26].

8.5 Software and LLM Supply Chain

Researchers have extensively studied the security and trust challenges in open-source software supply chains. Boughton [5] propose a framework for decomposing and measuring trust in open-source software supply chains by identifying factors that influence the trustworthiness of dependencies. From an adversarial perspective, Ladisa et al. [44] provide a systematic overview of attack techniques targeting open-source software supply chains. Complementary to these studies, Zahan et al. [107] evaluate security metrics and practices in open-source dependencies. Focusing on the JavaScript ecosystem, Zahan [108] analyze npm package metadata and propose weak-link signals to detect potential supply chain risks, while Miller [58] study how the npm ecosystem responds to dependency abandonment and its implications for supply chain sustainability. With the rapid growth of deep learning development, researchers have increasingly focused on the deep learning software supply chain. Tan et al. [19, 78] and Chen et al. [9] conduct exploratory studies on the supply chains of major deep learning frameworks (e.g., PyTorch and TensorFlow) to understand their downstream dependencies, development dynamics, collaboration patterns, and ecosystem differences. From another perspective, some studies focus on faults and failures in the deep learning stack. Huang et

al. [33] perform an empirical investigation of dependency-related bugs in the deep learning software stack. Jajal et al. [36] examine interoperability issues of ONNX model converters in deep learning frameworks, highlighting the challenges of cross-framework model portability.

Software Composition Analysis is widely leveraged to systematically identify software components, supporting tasks such as vulnerability detection [40, 114], license analysis [20], and supply chain risk assessment [61]. The Software Bill of Materials (SBOM) [59, 73, 106] provides a structured description of the internal components of a software system and their dependency relationships. Traditional SCA techniques focus on extracting components from artifacts at different levels. For example, explicit dependencies can be parsed from build configuration files (e.g., package.json and pom.xml) [34, 122]. Static code analysis can be used to identify calls to third-party libraries [19]. Binary feature matching can detect repackaged components [97]. With the rise of LLMs, the composition analysis has expanded beyond traditional elements to include models, datasets, frameworks, prompts, and external tools invoked during execution [32, 83]. The concept of SBOM has been extended to the AI domain, such as AI Bill of Materials (AI BOM) [1, 4] and ML Bill of Materials (ML BOM) [3]. At the standardization level, CycloneDX has extended support for ML BOM, and SPDX 3.0 has also begun to discuss and support the modeling of AI-related assets [2]. However, achieving transparent composition analysis for LLM-based systems presents substantial technical challenges. Prior studies have found that such models may memorize training data and leak parts of the training corpus [6, 76]. To infer training data membership, membership inference attacks (MIA) [75] have been adopted. Mattern et al. [57] propose generating contrastive samples that are semantically similar to the target sample to amplify the sample differences. Zhang et al. [113] infer membership by measuring the deviation between the word frequency distribution within a document and that of the overall corpus. Shi et al. [74] and Li et al. [47] propose the reference-free approaches based on the hypothesis that low-probability tokens tend to appear less frequently in member samples. Nevertheless, Duan et al. [13] found that the boundary between member and non-member samples is inherently fuzzy and continuous in the context of LLMs, leading to limited effectiveness. Besides, Ren et al. [69] studies the evolution of neural networks by constructing a large-scale neural network bill of materials database.

Researchers have extensively studied the security and trust challenges in open-source software supply chains. Boughton [5] propose a framework for decomposing and measuring trust in open-source software supply chains by identifying factors that influence the trustworthiness of dependencies. From an adversarial perspective, Ladisa et al. [44] provide a systematic overview of attack techniques targeting open-source software supply chains. Complementary to these studies, Zahan et al. [107] evaluate security metrics and practices in open-source dependencies. Focusing on the JavaScript ecosystem, Zahan [108] analyze npm package metadata and propose weak-link signals to detect potential supply chain risks, while Miller [58] study how the npm ecosystem responds to dependency abandonment and its implications for supply chain sustainability. With the rapid growth of deep learning development, researchers have increasingly focused on the deep learning software supply

chain. Tan et al.[19, 78] and Chen et al.[9] conduct exploratory studies on the supply chains of major deep learning frameworks (e.g., PyTorch and TensorFlow) to understand their downstream dependencies, development dynamics, collaboration patterns, and ecosystem differences. From another perspective, some studies focus on faults and failures in the deep learning stack. Huang et al.[33] perform an empirical investigation of dependency-related bugs in the deep learning software stack. Jajal et al.[36] examine interoperability issues of ONNX model converters in deep learning frameworks, highlighting the challenges of cross-framework model portability.

Software Composition Analysis is widely leveraged to systematically identify software components, supporting tasks such as vulnerability detection [40, 114], license analysis [20], and supply chain risk assessment [61]. The Software Bill of Materials (SBOM) [59, 73, 106] provides a structured description of the internal components of a software system and their dependency relationships. Traditional SCA techniques focus on extracting components from artifacts at different levels. For example, explicit dependencies can be parsed from build configuration files (e.g., package.json and pom.xml) [34, 122]. Static code analysis can be used to identify calls to third-party libraries [19]. Binary feature matching can detect repackaged components [97]. With the rise of LLMs, the composition analysis has expanded beyond traditional elements to include models, datasets, frameworks, prompts, and external tools invoked during execution [32, 83]. The concept of SBOM has been extended to the AI domain, such as AI Bill of Materials (AI BOM) [1, 4] and ML Bill of Materials (ML BOM) [3]. At the standardization level, CycloneDX has extended support for ML BOM, and SPDX 3.0 has also begun to discuss and support the modeling of AI-related assets [2]. However, achieving transparent composition analysis for LLM-based systems presents substantial technical challenges. Prior studies have found that such models may memorize training data and leak parts of the training corpus [6, 76]. To infer training data membership, membership inference attacks (MIA) [75] have been adopted. Mattern et al. [57] propose generating contrastive samples that are semantically similar to the target sample to amplify the sample differences. Zhang et al. [113] infer membership by measuring the deviation between the word frequency distribution within a document and that of the overall corpus. Shi et al. [74] and Li et al. [47] propose the reference-free approaches based on the hypothesis that low-probability tokens tend to appear less frequently in member samples. Nevertheless, Duan et al. [13] found that the boundary between member and non-member samples is inherently fuzzy and continuous in the context of LLMs, leading to limited effectiveness. Besides, Ren et al. [69] studies the evolution of neural networks by constructing a large-scale neural network bill of materials database.

9 Discussion

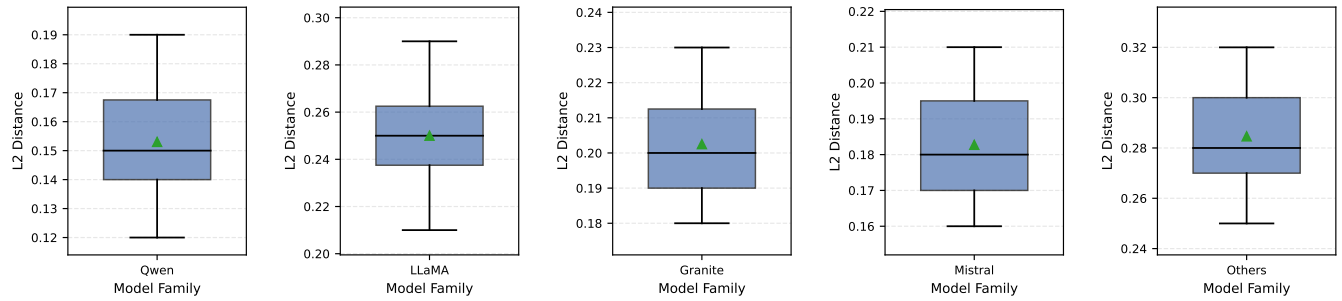
References

- [1] 2026. AI Bill Of Materials (AIBOM) Project. <https://owaspaibom.org/>. Accessed January 6, 2026.
- [2] 2026. Implementing an AI BOM. <https://spdx.dev/implementing-an-ai-bom/>. Accessed January 6, 2026.
- [3] 2026. Machine Learning Bill of Materials (ML-BOM). <https://cyclonedx.org/capabilities/mlbom/>. Accessed January 6, 2026.
- [4] Karen Bennet, Gopi Krishnan Rajbahadur, Arthit Suriyawongkul, and Kate Stewart. 2025. Implementing ai bill of materials (ai bom) with spdx 3.0: A comprehensive guide to creating ai and dataset bill of materials. *arXiv preprint arXiv:2504.16743* (2025).
- [5] Lina Boughton, Courtney Miller, Yasemin Acar, Dominik Wermke, and Christian Kästner. 2024. Decomposing and measuring trust in open-source software supply chains. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 57–61.
- [6] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*. 2633–2650.
- [7] Laurent Charette, Lingyang Chu, Yizhou Chen, Jian Pei, Lanjun Wang, and Yong Zhang. 2022. Cosine model watermarking against ensemble distillation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 9512–9520.
- [8] Ruibo Chen, Yihan Wu, Junfeng Guo, and Heng Huang. 2025. Improved unbiased watermark for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 20587–20601.
- [9] Yunqi Chen, Zhiyuan Wan, Yifei Zhuang, Ning Liu, David Lo, and Xiaohu Yang. 2025. Understanding the OSS communities of deep learning frameworks: a comparative case study of PyTorch and TensorFlow. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–30.
- [10] Zixuan Chen, Guangcong Wang, Jiahao Zhu, Jianhuang Lai, and Xiaohua Xie. 2025. GuardSplat: efficient and robust watermarking for 3d gaussian splatting. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 16325–16335.
- [11] Aloni Cohen, Alexander Hoover, and Gabe Schoenbach. 2025. Watermarking language models for many adaptive users. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2583–2601.
- [12] Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. 2022. Knowledge neurons in pretrained transformers. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8493–8502.
- [13] Michael Duan, Anshuman Suri, Nilofar Mireshghallah, Sewon Min, Weijia Shi, Luke Zettlemoyer, Yulia Tsvetkov, Yejin Choi, David Evans, and Hannaneh Hajishirzi. 2024. Do membership inference attacks work on large language models? *arXiv preprint arXiv:2402.07841* (2024).
- [14] Jianwei Fei, Zhihua Xia, Benedetta Tondi, and Mauro Barni. 2024. Wide flat minimum watermarking for robust ownership verification of gans. *IEEE Transactions on Information Forensics and Security* 19 (2024), 8322–8337.
- [15] Pierre Fernandez, Guillaume Couairon, Hervé Jégou, Matthijs Douze, and Teddy Furon. 2023. The stable signature: Rooting watermarks in latent diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 22466–22477.
- [16] Jiayi Fu, Xuandong Zhao, Ruihan Yang, Yuansen Zhang, Jiangjie Chen, and Yanghua Xiao. 2024. Gumbelsoft: Diversified language model watermarking via the gumbelmax-trick. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5791–5808.
- [17] Yu Fu, Deyi Xiong, and Yue Dong. 2024. Watermarking conditional text generation for ai detection: Unveiling challenges and a semantic-aware watermark remedy. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 18003–18011.
- [18] Guanhao Gan, Yiming Li, Dongxian Wu, and Shu-Tao Xia. 2023. Towards robust model watermark via reducing parametric vulnerability. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4751–4761.
- [19] Kai Gao, Runzhi He, Bing Xie, and Minghui Zhou. 2024. Characterizing deep learning package supply chains in pypi: Domains, clusters, and disengagement. *ACM Transactions on Software Engineering and Methodology* 33, 4 (2024), 1–27.
- [20] Daniel M German and Ahmed E Hassan. 2009. License integration patterns: Addressing license mismatches in component-based development. In *2009 IEEE 31st international conference on software engineering*. IEEE, 188–198.
- [21] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 5484–5495.
- [22] Jiyang Guan, Jian Liang, and Ran He. 2022. Are you stealing my model? sample correlation for fingerprinting deep neural networks. *Advances in Neural Information Processing Systems* 35 (2022), 36571–36584.
- [23] Martin Gubri, Dennis Ulmer, Hwaran Lee, Sangdo Yun, and Seong Joon Oh. 2024. Trap: Targeted random adversarial prompt honeypot for black-box identification. In *Findings of the Association for Computational Linguistics: ACL 2024*. 11496–11517.
- [24] Yiyang Guo, Ruizhe Li, Mude Hui, Hanzhong Guo, Chen Zhang, Chuangjian Cai, Le Wan, et al. 2024. Freqmark: Invisible image watermarking via frequency based optimization in latent space. *Advances in Neural Information Processing Systems* 37 (2024), 112237–112261.
- [25] Zhiwei He, Binglin Zhou, Hongkun Hao, Aiwei Liu, Xing Wang, Zhaopeng Tu, Zhuosheng Zhang, and Rui Wang. 2024. Can watermarks survive translation?

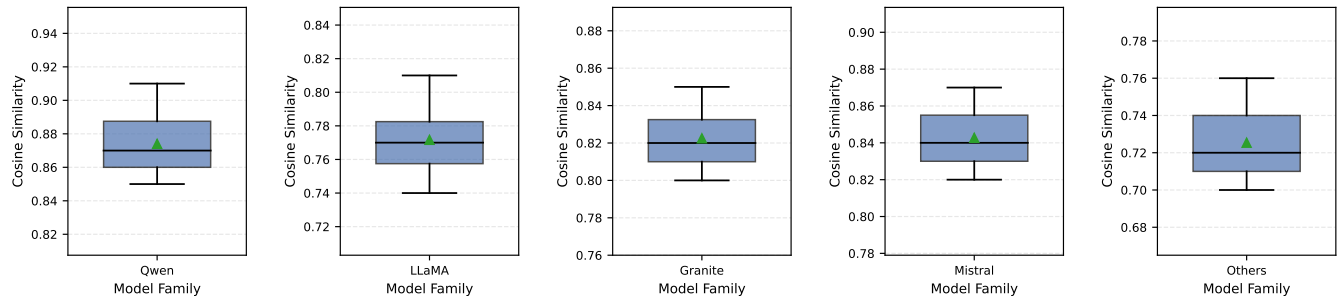
- on the cross-lingual consistency of text watermark for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4115–4129.
- [26] Eliahu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel Amar, and Yedid Hoshen. 2025. We Should Chart an Atlas of All the World's Models. *arXiv preprint arXiv:2503.10633* (2025).
- [27] Eliahu Horwitz, Asaf Shul, and Yedid Hoshen. 2024. Unsupervised model tree heritage recovery. *arXiv preprint arXiv:2405.18432* (2024).
- [28] Abe Hou, Jingyu Zhang, Yichen Wang, Daniel Khashabi, and Tianxing He. 2024. k-SemStamp: A clustering-based semantic watermark for detection of machine-generated text. In *Findings of the Association for Computational Linguistics: ACL 2024*. 1706–1715.
- [29] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [30] Runyi Hu, Jie Zhang, Yiming Li, Jiwei Li, Qing Guo, Han Qiu, and Tianwei Zhang. 2025. Videoshield: Regulating diffusion-based video generation models via watermarking. *arXiv preprint arXiv:2501.14195* (2025).
- [31] Huayang Huang, Yu Wu, and Qian Wang. 2024. Robin: Robust and invisible watermarks for diffusion models with adversarial optimization. *Advances in Neural Information Processing Systems* 37 (2024), 3937–3963.
- [32] Kaifeng Huang, Bihuan Chen, You Lu, Susheng Wu, Dingji Wang, Yiheng Huang, Haowen Jiang, Zhuotong Zhou, Junming Cao, and Xin Peng. 2024. Lifting the Veil on Composition, Risks, and Mitigations of the Large Language Model Supply Chain. *arXiv preprint arXiv:2410.21218* (2024).
- [33] Kaifeng Huang, Bihuan Chen, Susheng Wu, Junming Cao, Lei Ma, and Xin Peng. 2023. Demystifying dependency bugs in deep learning stack. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 450–462.
- [34] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.
- [35] Mingjia Huo, Sai Ashish Somayajula, Youwei Liang, Ruisi Zhang, Farinaz Koushanfar, and Pengtao Xie. 2024. Token-specific watermarking with enhanced detectability and semantic coherence for large language models. *arXiv preprint arXiv:2402.18059* (2024).
- [36] Purvish Jajal, Wenxin Jiang, Arav Tewari, Erik Kocinar, Joseph Woo, Anusha Sarraf, Yung-Hsiang Lu, George K Thiruvathukal, and James C Davis. 2024. Interoperability in deep learning: A user survey and failure analysis of onnx model converters. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1466–1478.
- [37] Youngdong Jang, Hyunje Park, Feng Yang, Heeju Ko, Euijin Choo, and Sangpil Kim. 2025. 3d-gsw: 3d gaussian splatting for robust watermarking. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 5938–5948.
- [38] Shengpeng Ji, Ziyue Jiang, Jialong Zuo, Minghui Fang, Yifu Chen, Tao Jin, and Zhou Zhao. 2025. Speech watermarking with discrete intermediate representations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24239–24247.
- [39] Zhe Ji, Qiansiqi Hu, Yicheng Zheng, Liyao Xiang, and Xinning Wang. 2024. A principled approach to natural language watermarking. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 2908–2916.
- [40] Ling Jiang, Junwen An, Huihui Huang, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2024. Binaryai: Binary software composition analysis via intelligent binary source code matching. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [41] Nikola Jovanović, Robin Staab, and Martin Vechev. 2024. Watermark stealing in large language models. *arXiv preprint arXiv:2402.19361* (2024).
- [42] Andre Kassis and Urs Hengartner. 2025. UnMarker: a universal attack on defensive image watermarking. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2602–2620.
- [43] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. In *International conference on machine learning*. PMLR, 17061–17084.
- [44] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. Sok: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1509–1526.
- [45] Peixuan Li, Pengzhou Cheng, Fangqi Li, Wei Du, Haodong Zhao, and Gongshen Liu. 2023. Plmmark: a secure and robust black-box watermarking framework for pre-trained language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 14991–14999.
- [46] Xingchi Li, Guanxun Li, and Xianyang Zhang. 2024. Segmenting watermarked texts from language models. *Advances in Neural Information Processing Systems* 37 (2024), 14634–14665.
- [47] Yuanheng Li, Zhuoyang Chen, Xiaoyun Liu, Yuhao Wang, Mingwei Liu, Yang Shi, Kaifeng Huang, and Shengjie Zhao. 2025. Uncovering Pretraining Code in LLMs: A Syntax-Aware Attribution Approach. *The 40th Annual AAAI Conference on Artificial Intelligence* (2025).
- [48] Yiming Li, Linghui Zhu, Xiaojun Jia, Yong Jiang, Shu-Tao Xia, and Xiaochun Cao. 2022. Defending against model stealing via verifying embedded external features. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 1464–1472.
- [49] Yiming Li, Mingyan Zhu, Xue Yang, Yong Jiang, Tao Wei, and Shu-Tao Xia. 2023. Black-box dataset ownership verification via backdoor watermarking. *IEEE Transactions on Information Forensics and Security* 18 (2023), 2318–2332.
- [50] Dongdong Lin, Benedetta Tondi, Bin Li, and Mauro Barni. 2024. A cyclegan watermarking method for ownership verification. *IEEE Transactions on Dependable and Secure Computing* 22, 2 (2024), 1040–1054.
- [51] Aiwei Liu, Leyi Pan, Xuming Hu, Shu'ang Li, Lijie Wen, Irwin King, and Philip S Yu. 2023. An unforgeable publicly verifiable watermark for large language models. *arXiv preprint arXiv:2307.16230* (2023).
- [52] Guan-Hong Liu, Tianrong Chen, Evangelos Theodorou, and Molei Tao. 2023. Mirror diffusion models for constrained and watermarked generation. *Advances in Neural Information Processing Systems* 36 (2023), 42898–42917.
- [53] Weizhi Liu, Yue Li, Dongdong Lin, Hui Tian, and Haizhou Li. 2024. Groot: Generating robust watermark for diffusion-model-based audio synthesis. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 3294–3302.
- [54] Yepeng Liu and Yuheng Bu. 2024. Adaptive text watermark for large language models. *arXiv preprint arXiv:2401.13927* (2024).
- [55] Nils Lukas and Florian Kerschbaum. 2023. {PTW}: Pivotal tuning watermarking for {Pre-Trained} image generators. In *32nd USENIX Security Symposium (USENIX Security 23)*. 2241–2258.
- [56] Ziyuan Luo, Anderson Rocha, Boxin Shi, Qing Guo, Haoliang Li, and Renjie Wan. 2025. The nerf signature: Codebook-aided watermarking for neural radiance fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [57] Justus Mattern, Fatemehsadat Miresghallah, Zhijing Jin, Bernhard Schölkopf, Minmaya Sachan, and Taylor Berg-Kirkpatrick. 2023. Membership inference attacks against language models via neighbourhood comparison. In *Findings of the Association for Computational Linguistics: ACL 2023*. 11330–11343.
- [58] Courtney Miller, Mahmoud Jahanshahi, Audris Mockus, Bogdan Vasilescu, and Christian Kastner. 2025. Understanding the response to open-source dependency abandonment in the npm ecosystem. (2025).
- [59] Mehdi Mirakhorli, Derek Garcia, Schuyler Dillon, Kevin Laporte, Matthew Morrison, Henry Lu, Viktoria Koscinski, and Christopher Enoch. 2024. A landscape study of open source and proprietary tools for software bill of materials (sbom). *arXiv preprint arXiv:2402.11151* (2024).
- [60] Andreas Müller, Denis Lukovnikov, Jonas Thietke, Asja Fischer, and Erwin Quiring. 2025. Black-box forgery attacks on semantic watermarks for diffusion models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 20937–20946.
- [61] Sabato Nocera, Sira Vegas, Giuseppe Scanniello, and Natalia Juristo Juzgado. 2025. Software Composition Analysis and Supply Chain Security in Apache Projects: an Empirical Study. *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)* (2025), 103–115.
- [62] Momose Oyama, Hiroaki Yamagiwa, Yusuke Takase, and Hidetoshi Shimodaira. 2025. Mapping 1,000+ Language Models via the Log-Likelihood Vector. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 32983–33038.
- [63] Leyi Pan, Aiwei Liu, Shiyu Huang, Yijian Lu, Xuming Hu, Lijie Wen, Irwin King, and Philip S Yu. 2025. Can LLM Watermarks Robustly Prevent Unauthorized Knowledge Distillation?. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13228–13251.
- [64] Kaiyi Pang, Tao Qi, Chuhan Wu, Minhao Bai, Minghu Jiang, and Yongfeng Huang. 2025. ModelShield: adaptive and robust watermark against model extraction attack. *IEEE Transactions on Information Forensics and Security* 20 (2025), 1767–1782.
- [65] Dario Pasquini, Evgenios M Kornaropoulos, and Giuseppe Ateniese. 2025. {LLMmap}: Fingerprinting for large language models. In *34th USENIX Security Symposium (USENIX Security 25)*. 299–318.
- [66] Zirui Peng, Shaofeng Li, Guoxing Chen, Cheng Zhang, Haojin Zhu, and Minhui Xue. 2022. Fingerprinting deep neural networks globally via universal adversarial perturbations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 13430–13439.
- [67] Jipeng Qiang, Shiyu Zhu, Yun Li, Yi Zhu, Yunhao Yuan, and Xindong Wu. 2023. Natural language watermarking via paraphraser-based lexical substitution. *Artificial Intelligence* 317 (2023), 103859.
- [68] Zihan Qiu, Zeyu Huang, Youcheng Huang, and Jie Fu. 2024. Empirical study on updating key-value memories in transformer feed-forward layers. *arXiv preprint arXiv:2402.12233* (2024).
- [69] Xiaoning Ren, Yuhang Ye, Xiongfei Wu, Yueming Wu, and Yinxing Xue. 2025. Demystifying the Evolution of Neural Networks with BOM Analysis: Insights from a Large-Scale Study of 55,997 GitHub Repositories. *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering* (2025).
- [70] Yubing Ren, Ping Guo, Yanan Cao, and Wei Ma. 2024. Subtle signatures, strong shields: Advancing robust and imperceptible watermarking in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*.

- 5508–5519.
- [71] Hadi Salman, Alaa Khaddaj, Guillaume Leclerc, Andrew Ilyas, and Aleksander Madry. 2023. Raising the cost of malicious ai-powered image editing. *arXiv preprint arXiv:2302.06588* (2023).
- [72] Tom Sander, Pierre Fernandez, Alain Durmus, Matthijs Douze, and Teddy Furon. 2024. Watermarking makes language models radioactive. *Advances in Neural Information Processing Systems* 37 (2024), 21079–21113.
- [73] Vandana Verma Sehgal and PS Ambili. 2023. A taxonomy and survey of software bill of materials (SBOM) generation approaches. In *Analytics Global Conference*. 40–51.
- [74] Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. 2023. Detecting pretraining data from large language models. *arXiv preprint arXiv:2310.16789* (2023).
- [75] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 3–18.
- [76] Congzheng Song and Vitaly Shmatikov. 2019. Auditing data provenance in text-generation models. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 196–206.
- [77] Zhensu Sun, Xiaoning Du, Fu Song, and Li Li. 2023. Codemark: Imperceptible watermarking for code datasets against neural code completion models. In *Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering*. 1561–1572.
- [78] Xin Tan, Kai Gao, Minghui Zhou, and Li Zhang. 2022. An exploratory study of deep learning supply chain. In *Proceedings of the 44th international conference on software engineering*. 86–98.
- [79] Shangqing Tu, Yuliang Sun, Yushi Bai, Jifan Yu, Lei Hou, and Juanzi Li. 2024. Waterbench: Towards holistic evaluation of watermarks for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1517–1542.
- [80] Betty Van Aken, Benjamin Winter, Alexander Löser, and Felix A Gers. 2019. How does bert answer questions? a layer-wise analysis of transformer representations. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1823–1832.
- [81] Lean Wang, Wenkai Yang, Deli Chen, Hao Zhou, Yankai Lin, Fandong Meng, Jie Zhou, and Xu Sun. 2023. Towards codable watermarking for injecting multi-bits information to LLMs. *arXiv preprint arXiv:2307.15992* (2023).
- [82] Suqing Wang, Ziyang Ma, Li Xinyi, and Zuchao Li. 2025. Ghost in the Transformer: Detecting Model Reuse with Invariant Spectral Signatures. *arXiv preprint arXiv:2511.06390* (2025).
- [83] Sheno Wang, Yanjie Zhao, Xinyi Hou, and Haoyu Wang. 2025. Large language model supply chain: A research agenda. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–46.
- [84] Yichen Wang, Shangbin Feng, Abe Hou, Xiao Pu, Chao Shen, Xiaoming Liu, Yulia Tsvetkov, and Tianxing He. 2024. Stumbling blocks: Stress testing the robustness of machine-generated text detectors under attacks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2894–2925.
- [85] Yuchen Wang, Xingyu Zhu, Guanhuai Ye, Shiyao Zhang, and Xuetao Wei. 2024. Achieving resolution-agnostic dnn-based image watermarking: A novel perspective of implicit neural representation. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 10354–10362.
- [86] Zilan Wang, Junfeng Guo, Jiacheng Zhu, Yiming Li, Heng Huang, Muhao Chen, and Zhengzhong Tu. 2025. Sleepmark: Towards robust watermark against fine-tuning text-to-image diffusion models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 8213–8224.
- [87] Zhenyi Wang, Yihan Wu, and Heng Huang. 2024. Defense against model extraction attack by bayesian active watermarking. In *Forty-first International Conference on Machine Learning*.
- [88] Yuxin Wen, John Kirchenbauer, Jonas Geiping, and Tom Goldstein. 2023. Tree-rings watermarks: Invisible fingerprints for diffusion images. *Advances in Neural Information Processing Systems* 36 (2023), 58047–58063.
- [89] Qilong Wu and Varun Chandrasekaran. 2024. Bypassing llm watermarks with color-aware substitutions. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8549–8581.
- [90] Yihan Wu, Zhengmian Hu, Junfeng Guo, Hongyang Zhang, and Heng Huang. 2023. A resilient and accessible distribution-preserving watermark for large language models. *arXiv preprint arXiv:2310.07710* (2023).
- [91] Zhaomin Wu, Haodong Zhao, Ziyang Wang, Jizhou Guo, Qian Wang, and Bingsheng He. 2025. LLM DNA: Tracing Model Evolution via Functional Representations. *arXiv preprint arXiv:2509.24496* (2025).
- [92] Zehao Wu, Yanjie Zhao, and Haoyu Wang. 2025. TensorGuard: Gradient-Based Model Fingerprinting for LLM Similarity Detection and Family Classification. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 445–456.
- [93] Zhiqiu Xu, Amish Sethi, Mayur Naik, and Ser-Nam Lim. 2025. Delta Activations: A Representation for Finetuned Large Language Models. *arXiv preprint arXiv:2509.04442* (2025).
- [94] Zhenhua Xu, Yiran Zhao, Mengting Zhong, Dezhong Kong, Changting Lin, Tong Qiao, and Meng Han. 2026. DNF: Dual-Layer Nested Fingerprinting for Large Language Model Intellectual Property Protection. *arXiv preprint arXiv:2601.08223* (2026).
- [95] Shojiro Yamabe, Futa Kai Waseda, Tsubasa Takahashi, and Koki Wataoka. 2025. MergePrint: Merge-Resistant Fingerprints for Robust Black-box Ownership Verification of Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6894–6916.
- [96] Borui Yang, Wei Li, Liyao Xiang, and Bo Li. 2024. Srcmarker: Dual-channel source code watermarking via scalable code transformations. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4088–4106.
- [97] Can Yang, Zhengzi Xu, Hongxu Chen, Yang Liu, Xiaorui Gong, and Baoxu Liu. 2022. Modx: binary level partially imported third-party library detection via program modularization and semantic matching. In *Proceedings of the 44th International Conference on Software Engineering*. 1393–1405.
- [98] Kang Yang, Run Wang, and Lina Wang. 2022. MetaFinger: Fingerprinting the Deep Neural Networks with Meta-training.. In *IJCAI*. 776–782.
- [99] Yuxin Yang, Qiang Li, Yuan Hong, and Binghui Wang. 2024. Fedgmark: Certifiably robust watermarking for federated graph learning. *Advances in Neural Information Processing Systems* 37 (2024), 48971–48995.
- [100] Zijin Yang, Kai Zeng, Kejiang Chen, Han Fang, Weiming Zhang, and Nenghai Yu. 2024. Gaussian shading: Provable performance-lossless image watermarking for diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12162–12171.
- [101] Hongwei Yao, Jian Lou, Zhan Qin, and Kui Ren. 2024. Promptcare: Prompt copyright protection by watermark injection and verification. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 845–861.
- [102] Nicolas Yax, Pierre-Yves Oudeyer, and Stefano Palminteri. 2024. PhyloLM: Inferring the phylogeny of large language models and predicting their performances in benchmarks. *arXiv preprint arXiv:2404.04671* (2024).
- [103] KiYoon Yoo, Wonhyuk Ahn, Jiho Jang, and Nojun Kwak. 2023. Robust multi-bit natural language watermarking through invariant features. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2092–2115.
- [104] Do-hyeon Yoon, Minsoo Chun, Thomas Allen, Hans Müller, Min Wang, and Rajesh Sharma. 2025. Intrinsic Fingerprint of LLMs: Continue Training is NOT All You Need to Steal A Model! *arXiv preprint arXiv:2507.03014* (2025).
- [105] Rungeng Yu and Xinchao Wang. 2024. Neural lineage. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4797–4807.
- [106] Sheng Yu, Wei Song, Xunchao Hu, and Heng Yin. 2024. On the correctness of metadata-based SBOM generation: A differential analysis approach. In *2024 54th Annual IEEE/IFIP International conference on dependable systems and networks (DSN)*. 29–36.
- [107] Nusrat Zahan. 2023. Software supply chain risk assessment framework. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 251–255.
- [108] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 331–340.
- [109] Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. 2024. Huref: Human-readable fingerprint for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 126332–126362.
- [110] Jie Zhang, Dongdong Chen, Jing Liao, Zehua Ma, Han Fang, Weiming Zhang, Huamin Feng, Gang Hua, and Nenghai Yu. 2024. Robust model watermarking for image processing networks via structure consistency. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 10 (2024), 6985–6992.
- [111] Jie Zhang, Dongrui Liu, Chen Qian, Linfeng Zhang, Yong Liu, Yu Qiao, and Jing Shao. 2024. Reef: Representation encoding fingerprints for large language models. *arXiv preprint arXiv:2410.14273* (2024).
- [112] Ruichong Zhang. 2025. Matrix-driven instant review: Confident detection and reconstruction of llm plagiarism on pc. *arXiv e-prints* (2025), arXiv–2508.
- [113] Weichao Zhang, Ruqing Zhang, Jiafeng Guo, Maarten de Rijke, Yixing Fan, and Xueqi Cheng. 2024. Pretraining data detection for large language models: A divergence-based calibration method. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 5263–5274.
- [114] Lida Zhao, Sen Chen, Zhengzi Xu, Chengwei Liu, Lyuye Zhang, Jiahui Wu, Jun Sun, and Yang Liu. 2023. Software composition analysis for vulnerability detection: An empirical study on Java projects. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 960–972.
- [115] Xuandong Zhao, Sam Gunn, Miranda Christ, Jaiden Fairouz, Andres Fabrega, Nicholas Carlini, Sanjam Garg, Sanghyun Hong, Milad Nasr, Florian Tramer, et al. 2025. Sok: Watermarking for ai-generated content. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2621–2639.

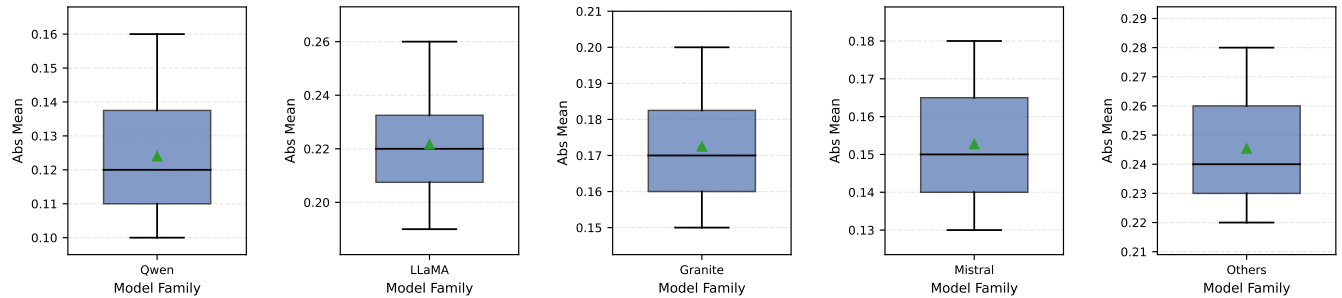
- [116] Xuandong Zhao, Chenwen Liao, Yu-Xiang Wang, and Lei Li. 2025. Efficiently identifying watermarked segments in mixed-source texts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6304–6316.
- [117] Xuandong Zhao, Yu-Xiang Wang, and Lei Li. 2023. Protecting language generation models via invisible watermarking. In *International Conference on Machine Learning*. PMLR, 42187–42199.
- [118] Haonan Zhong, Jiamin Chang, Ziyue Yang, Tingmin Wu, Pathum Chamikara Mahawaga Arachchige, Chehara Pathmabandu, and Minhui Xue. 2023. Copyright protection and accountability of generative ai: Attack, watermarking and attribution. In *Companion Proceedings of the ACM Web Conference 2023*. 94–98.
- [119] Tong Zhou, Xuandong Zhao, Xiaolin Xu, and Shaolei Ren. 2024. Bileve: Securing text provenance in large language models against spoofing with bi-level signature. *Advances in Neural Information Processing Systems* 37 (2024), 56054–56075.
- [120] Yichu Zhou and Vivek Srikumar. 2022. A closer look at how fine-tuning changes BERT. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1046–1061.
- [121] Hongyu Zhu, Sichu Liang, Wentao Hu, Li Fangqi, Ju Jia, and Shi-Lin Wang. 2024. Reliable model watermarking: Defending against theft without compromising on evasion. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 10124–10133.
- [122] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small world with high risks: A study of security threats in the npm ecosystem. In *28th USENIX Security symposium (USENIX security 19)*. 995–1010.
- [123] Wei Zong, Yang-Wai Chow, Willy Susilo, Joonsang Baek, and Seyit Camtepe. 2025. {AudioMarkNet}: Audio watermarking for deepfake speech detection. In *34th USENIX Security Symposium (USENIX Security 25)*. 4663–4682.



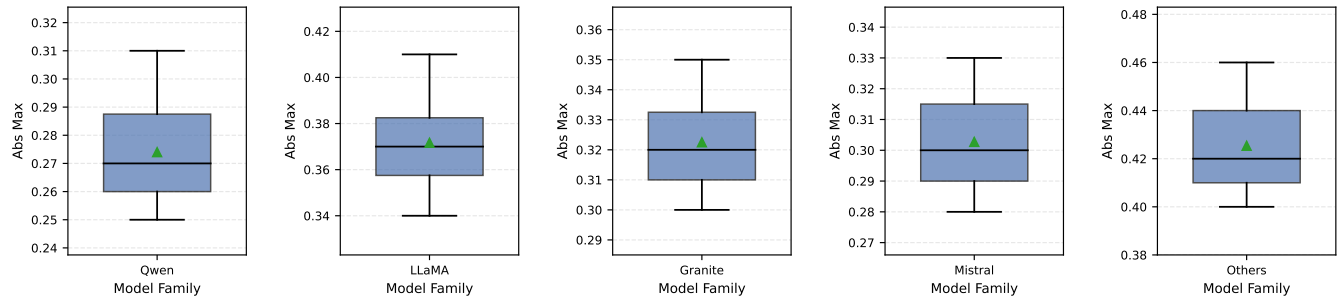
(a) L2 Distance



(b) Cosine Similarity



(c) Abs Mean



(d) Abs Max

